University of Piraeus
Department of Digital Systems

Mobile and Wireless Communications Security

ΟΝΟΜΑΤΕΠΩΝΥΜΟ	ΑΡΙΘΜΟΣ ΜΗΤΡΩΟΥ	EMAIL	ΗΜΕΡΟΜΗΝΙΑ
ELVIS AVDULI	E22001	avdulielvis@gmail.com	30/04/2026

8th Semester

Table of Contents

1. Assignment Description.....2
2. Hardening of UnsafeBank Contract.....2
3. Hardening of Victim Contract 3
4. Hardening of withdraw_1 Contract.....5
5. Hardening of withdraw_2 Contract.....6

1. Assignment Description

The objective of this assignment is to analyze a set of vulnerable smart contracts and identify potential security issues that may arise from improper implementation. The contracts are deployed and tested within a blockchain environment, using Remix.

The main goal of the auditing process is to examine how each contract operates, detect logical security flaws and propose appropriate mitigation techniques. Special attention will be given to vulnerabilities related to access control, incorrect balance management, and unsafe arithmetic operations. These issues are critical in blockchain systems, as smart contracts are immutable and cannot be easily corrected once deployed.

In addition to finding those vulnerabilities, this report focuses on proposing "hardening" techniques. Hardening is the process of modifying the contract to make it more secure, reliable and resistant to malicious attacks.

2. Hardening of UnsafeBank Contract

The UnsafeBank contract contains severe vulnerabilities mainly related to missing access control and incorrect balance handling. The contract allows users to deposit funds on behalf of other addresses without verifying the actual Ether sent, which leads to inconsistencies between recorded balances and real funds. More specifically, the deposit function trusts a user-defined value parameter instead of using msg.value, which means that the internal accounting of the contract may not reflect the real amount of Ether deposited. This is a serious design flaw because the contract can store false balances.

However, the most critical issue lies in withdraw function, where any user can withdraw funds from any address without authorization. This behavior completely breaks the security model of the contract, as it allows attackers to manipulate balances freely. In a secure implementation, only the owner of the funds should be allowed to perform withdrawals. To fix this issue, access control must be enforced by validating the caller of the function. Specifically, the contract must ensure that the address requesting the withdrawal matches the owner of the funds.

In the hardened version of the contract, the deposit function was simplified so that users can only deposit funds for themselves, and the amount recorded is taken

directly from msg.value. This guarantees consistency between the blockchain transaction and theinternal state of the contract. In addition, the withdrawal function was redesigned so that it no longer accepts an external address as a parameter. Instead, it operates only on msg.sender, which ensures that users can withdraw only their own balances.

Another important improvement is that the original contract did not actually transfer Ether during withdrawal but only modified the balance mapping. In the secure version, this was corrected by subtracting the requested amount from the sender using msg.sender.transfer(amount). As a result, the new contract is both logically correct and significantly more secure.

```
1   pragma solidity 0.4.17;
2
3   contract SafeBank {
4       mapping(address => uint256) public balances;
5
6       function deposit() public payable {    20453 gas
7           balances[msg.sender] += msg.value;
8       }
9
10      function withdraw(uint256 amount) public {    infinite gas
11          require(balances[msg.sender] >= amount);
12          balances[msg.sender] -= amount;
13          msg.sender.transfer(amount);
14      }
15  }
```

3. Hardening of Victim Contract

The Victim contract presents several security issues that stem primarily from improper validation of the user input and the absence of access control mechanisms. The most critical vulnerability appears in the withdraw function, which allows any user to specify both the amount to be withdrawn and the address from which the balance will be deducted. This introduces a serious flaw, as it enables malicious users to manipulate the balances of other accounts without any form of authorization.

In particular, the contract does not verify whether the caller of the function is the owner of the funds. As a result, an attacker can call the withdraw function using another user's address as the source of the funds. In a secure system, each user should only be able to manage their own funds.

Another important issue is related to insufficient balance validation. The contract checks only whether the balance is greater than zero, but it does not ensure that the requested withdrawal amount is less or equal to the available balance. This can lead to incorrect state updates and potentially allow unintended behavior.

In the hardened version of the contract, the withdrawal function was redesigned to remove the ability to specify an external address. Instead, the function now operates exclusively on msg.sender, ensuring that users can only withdraw their own funds. This enforces proper access control and prevents unauthorized interactions with other users' balances.

Additionally, proper validation was introduced to ensure that the withdrawal amount does not exceed the user's available balance. The condition was corrected to verify that the balance is greater than or equal to the requested amount, allowing users to withdraw their full balance when needed. This guarantees that the contract maintains a consistent and valid state after each operation. Finally, the withdrawal logic was improved so that it not only updates the internal balance mapping but also transfers the corresponding amount of Ether to the user, ensuring correct and secure financial behavior.

```
pragma solidity ^0.4.17;

contract N0t_Victim {
  mapping (address => uint) public balances;

function deposit() public payable {    20453 gas
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint value) public {    infinite gas
        uint bal = balances[msg.sender];
        require(bal >= value);
        balances[msg.sender] -= value;
        msg.sender.transfer(value);
    }
}
```

4. Hardening of withdraw_1 Contract

The first contract presents several logical issues that make it unusable as a financial system. The most critical flaw is not related to access control, but to incorrect design. The contract initializes a fixed value of 0 ether and uses this value in the setBalance function, which results in every user having a balance of zero. As a consequence, the withdraw function always returns zero, making the contract non-functional. Additionally, the withdrawal function only returns a stored value instead of transferring Ether, meaning that no real funds are ever sent back to the user. This demonstrates that the contract fails to implement a valid deposit and withdrawal mechanism.

Also, the contract is missing the transfer command needed to move funds. On top of that, it uses a global variable (amount) to store the withdrawal value. This wastes a lot of gas in every transaction. Finally, the setBalance function uses a fixed value of 0 ether, so no real money can ever be deposited or tracked.

In the hardened version of the contract, the withdrawal function was changed to include the msg.sender.transfer(amount) command. This makes sure that Ether is safely sent back to the user. The amount variable was also changed to a local variable to save gas.

Additionally, the setBalance function was fixed by making it payable and using msg.value. This allows users to deposit real Ether. The withdrawal function now also checks if the user has enough balance before completing the action.

```
pragma solidity 0.4.17;

contract withdrawBalance {
    mapping (address => uint) balance;

    function setBalance() public payable {    20409 gas
        balance[msg.sender] += msg.value;
    }

    function withdraw() public returns (uint) {    infinite gas
        uint amount = balance[msg.sender];
        require(amount > 0);
        balance[msg.sender] = 0;
        msg.sender.transfer(amount);
        return amount;
    }
}
```

5. Hardening of withdraw_2 Contract

The second contract presents critical security issues that come from a lack of access control and bad starting values. The biggest vulnerability is in both the setBalance and withdraw functions, which allow any user to input a _victim address. This is a huge flaw because it lets malicious users change or reset the balances of other users without any permission.

Another critical issue is related to the initialization of the contract with an extremely large numerical value using the expression 2**256 - 1 ether. While this does not immediately cause an overflow in Solidity, it represents a value very close to the maximum limit of the unsigned integer type. This creates a dangerous situation when combined with the lack of access control, because it allows arbitrary assignment of very large balances. As a result, attackers can manipulate the contract state by assigning unrealistic amounts of funds to themselves or other

users. The real vulnerability, therefore, lies in the combination of unsafe initialization and unrestricted access to balance-modifying functions.

In the hardened version of the contract, the functions were changed to remove the _victim address. Instead, the contract now only uses msg.sender. This ensures that users can only control and withdraw their own funds. This fixes the access control issue and unauthorized users can no longer interfere with the balances of others.

Finally, the dangerous overflow value was removed. The code was updated to use msg.value for deposits and real transfer commands for withdrawals. This guarantees that the contract works correctly and users can safely manage their own assets.

```
pragma solidity 0.4.17;

contract withdrawBalance {
    mapping (address => uint) balance;

    function setBalance() public payable {      20409 gas
      balance[msg.sender] += msg.value;
    }

    function withdraw() public returns (uint) {    infinite gas
        uint amount = balance[msg.sender];
        require(amount > 0);
        balance[msg.sender] = 0;
        msg.sender.transfer(amount);
        return amount;
    }
}
```